

Simplified SSH access via container VPNs

Rohit Goswami · 2025-05-11 · ramblings · setup · remote

Using containers to SSH into intranet machines without routing all traffic through a VPN

Background

Most academic or corporate networks gate access to internal resources (like clusters or services) via their local intranet, requiring a VPN connection (e.g., using Cisco AnyConnect, FortiGate). Connecting to such a VPN on a personal machine typically routes all your internet traffic through that VPN. This is useful for certain tasks like accessing paywalled journal articles via university subscriptions, but it becomes problematic when:

- You only want some of your traffic to go through the VPN.
- You need to connect to multiple distinct services, each behind its own separate VPN. Managing these simultaneously or switching between them on a host system is often cumbersome or impossible.

These considerations affect interactive SSH sessions and complicate using tools for remote file management or other services that need to operate through these VPNs.

Tools

We'll be using [Podman](#) with [OpenConnect](#) and [OpenSSH](#) to build a VPN jumphost for simplified [ProxyJump](#) configurations and [rclone](#) for SFTP mounting.

Normal usage

To recap, the standard workflow, for say, an `anyconnect` VPN is as follows:

```
sudo openconnect $VPN -u $VUSER --useragent AnyConnect
# Another terminal
ssh $LMACHINE
```

Where `LMACHINE` is normally only accessible from the intranet, access to which is provided by `VPN` authenticated for `VUSER`. `sudo` is there since the networking of the current machine, say `CMACHINE` is affected. This works, but as noted has the side effect of routing the entire machine's traffic through the VPN. Trying to manage multiple such VPNs is difficult.

Caveats

Full Network Routing

Your entire machine's internet usage goes through the VPN.

Multiple VPNs

Switching between VPNs is slow, and running multiple simultaneously is generally not feasible.

Simpler Access

Even with one VPN, accessing many internal machines via direct SSH can be made more convenient with a jumphost and the SSH ProxyJump feature.

Container setup

Containerization is a powerful solution by isolating the VPN connection within a dedicated environment, leaving your host system's networking untouched. This container can then act as a secure SSH jumphost.

A basic container using an Alpine base with a non-root user for this is basically:

```

FROM alpine:latest

LABEL maintainer="rgoswami[at]ieee[dot]org"
LABEL description="Alpine jumphost: OpenConnect (interactive) as main process, OpenSSH server for ProxyJump."

ARG SSH_USER_NAME=jumphostuser
# It's safer to pass the public key content at build time than to have a default here that might be forgotten.
ARG USER_PUBLIC_KEY
ARG PASS=nothing

# Install necessary packages
RUN apk add --no-cache \
    tini \
    openconnect \
    openssh \
    openssh-server \
    bash \
    ca-certificates

# Create a non-root user for SSH connections INTO the container
# The password isn't really ever required, but causes an error without it
RUN adduser -g "${SSH_USER_NAME}" -D -s /bin/bash "${SSH_USER_NAME}" && \
    echo "${SSH_USER_NAME}:${(openssl passwd -1 $PASS)}" | chpasswd

# Setup SSH for this user using the public key provided at build time
RUN mkdir -p "/home/${SSH_USER_NAME}/.ssh" && \
    chmod 700 "/home/${SSH_USER_NAME}/.ssh" && \
    echo "${USER_PUBLIC_KEY}" > "/home/${SSH_USER_NAME}/.ssh/authorized_keys" && \
    chmod 600 "/home/${SSH_USER_NAME}/.ssh/authorized_keys" && \
    chown -R "${SSH_USER_NAME}:${SSH_USER_NAME}" "/home/${SSH_USER_NAME}/.ssh"

# Configure SSHD for security and ProxyJump functionality
RUN sed -i 's/^#?PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config && \
    sed -i 's/^#?PasswordAuthentication.*/PasswordAuthentication yes/' /etc/ssh/sshd_config && \
    sed -i 's/^#?PubkeyAuthentication.*/PubkeyAuthentication yes/' /etc/ssh/sshd_config && \
    \
    # Robustly set AllowTcpForwarding to yes
    sed -i '/^#?AllowTcpForwarding.*/d' /etc/ssh/sshd_config && \
    echo "AllowTcpForwarding yes" >> /etc/ssh/sshd_config && \
    \
    sed -i '/^#?AllowAgentForwarding.*/d' /etc/ssh/sshd_config && \
    echo "AllowAgentForwarding yes" >> /etc/ssh/sshd_config && \
    \
    sed -i '/^#?GatewayPorts.*/d' /etc/ssh/sshd_config && \
    echo "GatewayPorts no" >> /etc/ssh/sshd_config && \
    \
    sed -i '/^#?UsePAM.*/d' /etc/ssh/sshd_config && \
    echo "UsePAM no" >> /etc/ssh/sshd_config && \
    echo "ChallengeResponseAuthentication no" >> /etc/ssh/sshd_config && \
    \
    echo "ClientAliveInterval 60" >> /etc/ssh/sshd_config && \
    echo "ClientAliveCountMax 3" >> /etc/ssh/sshd_config && \
    echo "LogLevel DEBUG3" >> /etc/ssh/sshd_config

# Generate SSH host keys if they don't exist
RUN ssh-keygen -A

# Expose the SSH port
EXPOSE 22

# Copy the entrypoint script into the image
COPY entrypoint.sh /usr/local/bin/entrypoint.sh
RUN chmod +x /usr/local/bin/entrypoint.sh

# Use tini to manage the entrypoint script. The script itself runs as root.

```

```
ENTRYPOINT ["/sbin/tini, "--"]  
CMD ["/usr/local/bin/entrypoint.sh"]
```

Normally, I'm a fan of using `tinyssh` for basic containers but since this is meant to be a jump server, we need `AllowTcpForwarding` and so `openssh` is simpler. Combined with this `entrypoint.sh`:

```
#!/usr/bin/env sh  
  
# Exit immediately if a command exits with a non-zero status  
set -e  
  
echo "Starting SSH daemon..."  
/usr/sbin/sshd -e  
  
echo "SSH daemon active. Starting OpenConnect interactively..."  
echo "You will be prompted for your VPN password."  
  
# Now, execute OpenConnect in the foreground.  
# This script runs as root by default in Docker, which OpenConnect needs  
# to modify network routes and create the tun interface.  
# The --script flag is vital for DNS/routing updates.  
exec openconnect sample.vpnhost \  
-u me@sample.vpnhost \  
--useragent='AnyConnect' \  
--script=/etc/vpnc/vpnc-script
```

Here the command is hardcoded, but it can be adapted or parameterized via an environment variable if required.

Built with:

```
# Remember to modify this for your key  
export PUB_KEY_CONTENT=$(cat ~/.ssh/cstuff.pub)  
# Build the container called vpn-smart-jumphost  
docker build \  
--build-arg USER_PUBLIC_KEY="$PUB_KEY_CONTENT" \  
--build-arg SSH_USER_NAME=jumphostuser \  
--pull \  
-t vpn-smart-jumphost:latest .
```

Which is subsequently run via:

```
docker run --rm -it \  
--name vpn-smart-jump-container \  
--cap-add=NET_ADMIN \  
--device=/dev/net/tun:/dev/net/tun \  
-p 127.0.0.1:2200:22 \  
vpn-smart-jumphost:latest
```

This will prompt in the terminal for the VPN credentials, including an MFA if necessary. The network within the container is now correctly connected through to the VPN. Remember to keep this open as it maintains the VPN session (perhaps using `tmux`).

ProxyJump settings

For the SSH configuration now,

```
Host vpn-docker-jump
  HostName 127.0.0.1
  Port 2200
  User jumphostuser
  IdentitiesOnly yes
  IdentityFile ~/.ssh/cstuff

Host *.vpnhost !vpnhost
  User rgoswami
  ProxyJump vpn-docker-jump

Host someone.cstuff
  Hostname something.vpnhost
  User rgoswami
  ProxyJump vpn-docker-jump
```

Replace `~/.ssh/cstuff` with the path to the private key that corresponds to the public key you used during the build. Replace `rgoswami` with your username on the target systems if it's different.

Usage

Once the container is running, the VPN is connected within it, and your `~/.ssh/config` is set up, you can SSH to your remote machines from a new terminal window on your host:

```
ssh someone.cstuff
```

Or indeed, anything which matches `*.vpnhost`:

```
ssh some-other-server.vpnhost
```

The first time you connect via the jumphost, you may be asked to verify and accept the SSH host key of the container.

Mounting SFTP Remotes

The `ProxyJump` setup in `~/.ssh/config` is excellent for interactive SSH sessions, `scp`, and direct `sftp` commands. However, for `rclone` ([homepage](#)), which can mount a remote SFTP share, some more work is necessary for the VPN jumphost.

`rclone` can leverage SSH capabilities, but it does not parse directives from the global SSH config directly.

We can achieve this by constructing an appropriate `ProxyCommand` and passing it to `rclone` via its SFTP backend options.

```
ssh -o "ProxyCommand ssh jumphostuser@127.0.0.1 -p 2200 -i ~/.ssh/cosmolab -o IdentitiesOnly=yes -W %h:%p"
$USER@$VPN_HOSTNAME
```

The `ProxyCommand` effectively tells SSH how to connect to the intermediate jumphost first, and then tunnel the connection to the final SFTP server.

Essentially we have `rclone config edit` (followed by `n` and then `SFTP`) to end up with:

type

```
sftp
```

host

```
$RCLONE_REMOTE
```

user

```
$USER
```

pubkey_file

```
$SSH_KEY
```

key_use_agent

```
true
```

ssh

```
ssh -o "ProxyCommand ssh jumphostuser@127.0.0.1 -p 2200 -i ~/.ssh/cosmolab -o  
IdentitiesOnly=yes -W %h:%p" $USER@$VPN_HOSTNAME
```

Final usage proceeds in the usual fashion:

```
rclone mount "$RCLONE_REMOTE:/home/$USER" ~/mnt/somevpn --vfs-cache-mode full
```

Where `--vfs-cache-mode full` improves performance by caching files.

Conclusions

This containerized approach provides an isolated VPN connection and a convenient `ProxyJump` setup. This allows direct, targeted access to remote networks without impacting your entire host's networking. Remote file access via `rclone` also works well, which means integration with `emacs` and other IDEs is a snap.

A more solid variant of this basic concept developed into the [vpn-proxyjump](#) project.